

Concurrency and Distributed Systems

Week 4 – Part 2: `volatile` and Happens-Before

Dr. Mohamad Aoude

Faculty of Engineering

May 11, 2026

Part 2 Roadmap

- 1 From Visibility Problem to Visibility Tool
- 2 The Meaning of volatile
- 3 Visibility and Ordering
- 4 Happens-Before
- 5 Publication Patterns
- 6 Why volatile Is Not Atomic
- 7 Lab Preparation

Where We Are in Week 4

Part 1

We learned that unsynchronized shared variables are not reliable communication channels:

- stale reads can happen
- threads may observe different values
- the Java Memory Model defines what is allowed

Part 2

Now we study the first explicit memory-visibility tool:

`volatile`

and the key formal relation:

happens-before

Learning Objectives

By the end of this lecture, students should be able to:

- explain what `volatile` guarantees
- explain what `volatile` does *not* guarantee
- distinguish visibility from atomicity
- explain, in practical terms, the meaning of **happens-before**
- identify simple publication patterns using a `volatile` flag
- connect `volatile` to ordering constraints
- understand why `volatile` cannot fix `counter++`

Return to the Stop-Flag Problem

```
1      class StopDemo {
2          static boolean running = true;
3
4          public static void main(String[] args) throws
5              Exception {
6              Thread worker = new Thread(() -> {
7                  while (running) {
8                      // busy wait
9                  }
10                 System.out.println("Worker stopped");
11             });
12
13             worker.start();
14             Thread.sleep(1000);
15             running = false;
16         }
17     }
```

Fixing the Communication Variable

```
1      class StopDemoFixed {
2          static volatile boolean running = true;
3
4          public static void main(String[] args) throws
5              Exception {
6              Thread worker = new Thread(() -> {
7                  while (running) {
8                      // busy wait
9                  }
10                 System.out.println("Worker stopped");
11             });
12
13             worker.start();
14             Thread.sleep(1000);
15             running = false;
16         }
17     }
```

What volatile Means

Practical definition

A `volatile` variable is a shared variable whose reads and writes participate in stronger memory-visibility and ordering rules.

When one thread writes a new value to a `volatile` variable:

- other threads are required to observe that write correctly through later `volatile` reads
- certain reorderings around that access are restricted

Important

`volatile` is not a lock. It does not make threads take turns.

The Two Core Guarantees of volatile

1. Visibility

A write to a volatile variable by one thread becomes visible to later reads of that same variable by another thread.

2. Ordering

Operations around volatile accesses are constrained so that communication through the variable has meaningful sequence.

Simple memory role

A volatile variable acts as a controlled communication point between threads.

What volatile Does Not Guarantee

Critical limitation

volatile does **not** provide mutual exclusion.

So volatile does not make compound actions atomic, such as:

- incrementing a counter
- checking then acting
- updating multiple variables as one unit

Key sentence

volatile helps threads **see** updates. It does not make complex updates **indivisible**.

Visibility Example

```
1      class VisibilityFlag {
2          static volatile boolean ready = false;
3
4          public static void main(String[] args) throws
5              Exception {
6              Thread t = new Thread(() -> {
7                  while (!ready) {
8                      System.out.println("Observed ready =
9                          true");
10                 }
11
12                 t.start();
13                 Thread.sleep(1000);
14                 ready = true;
15             }
16         }
```

Ordering Intuition

Why ordering matters

Even if a thread eventually sees a value, we must also reason about *which earlier actions become visible together with it*.

Suppose a writer does:

- 1 prepare some data
- 2 set a flag saying “data is ready”

If the reader sees the flag, it should also see the prepared data in the intended order.

This is why visibility alone is not enough

Correct communication needs both:

- visibility
- ordering guarantees

Publication Through a Volatile Flag

```
1      class PublicationDemo {
2          static int data = 0;
3          static volatile boolean ready = false;
4
5          public static void main(String[] args) {
6              Thread writer = new Thread(() -> {
7                  data = 42;
8                  ready = true;
9              });
10
11              Thread reader = new Thread(() -> {
12                  while (!ready) {
13                      }
14                  System.out.println(data);
15              });
16
17              reader.start();
18          }
19      }
```

Why the Publication Pattern Works

Writer thread:

- ① writes `data = 42`
- ② then writes `ready = true`

Reader thread:

- ① spins until it sees `ready == true`
- ② then reads `data`

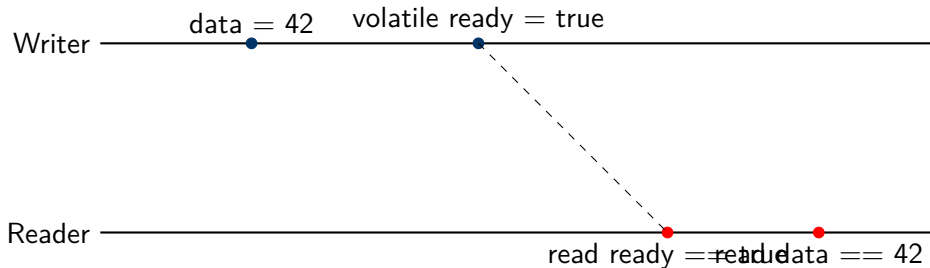
Key meaning

If the reader observes the volatile write to `ready`, Java guarantees the visibility of the relevant earlier write to `data`.

This is the beginning of safe publication reasoning

A volatile flag can publish the state prepared before that flag was written.

A Visual View of Publication



Interpretation

The volatile flag acts as the communication gateway that carries visibility and ordering effects.

Enter Happens-Before

Practical definition

If action A **happens-before** action B, then B is guaranteed to observe the effects of A in the way defined by the Java Memory Model.

This is one of the most important ideas in Java concurrency.

Think of it as

Not simply *earlier in clock time*, but *earlier with a guaranteed memory relationship*.

Clock Time Is Not Enough

Common misconception

Students often think:

If A happened first in real time, then B must see it.

That is false in concurrent systems.

Correct statement

What matters is not only temporal order, but whether the program established a valid **happens-before** relation.

Why this matters

Two actions may be separated in time and still lack the visibility guarantee needed for correct communication.

Important Happens-Before Rules

Some key practical rules are:

- A write to a volatile variable happens-before a later read of that same variable
- A call to `Thread.start()` happens-before actions in the started thread
- All actions in a thread happen-before another thread successfully returns from `join()`
- Unlocking a monitor happens-before a later lock of that same monitor

Connection to Week 3

We are not re-teaching locks here, but note that lock/unlock also establish memory guarantees.

Volatile Happens-Before Example

```
1      class HbExample {
2          static int value = 0;
3          static volatile boolean published = false;
4
5          public static void main(String[] args) {
6              Thread writer = new Thread(() -> {
7                  value = 99;
8                  published = true;
9              });
10
11              Thread reader = new Thread(() -> {
12                  while (!published) {
13                      }
14                  System.out.println(value);
15              });
16
17              reader.start();
18              writer.start();
19          }
20      }
```

What Happens-Before Gives You Here

In the previous example, once the reader observes:

```
published == true
```

it is not just seeing the flag itself.

It is also guaranteed to see the relevant earlier writes that logically preceded that volatile write, including:

```
value = 99
```

This is the power of happens-before

It provides a defined memory relationship, not just a guessed runtime ordering.

volatile as a Communication Signal

A useful student mental model is:

“ordinary data” + “volatile signal”

Pattern

- write ordinary shared state first
- then write a volatile flag
- another thread reads the volatile flag
- then safely consumes the prepared state

This is not universal thread safety

It is a very specific pattern for controlled communication.

What volatile Is Good For

Typical good uses include:

- stop flags
- readiness flags
- publication indicators
- one-writer / many-reader visibility patterns
- simple state variables where no compound action is required

Engineering rule

Use `volatile` when the main problem is:

“Other threads must see this update correctly.”

not when the problem is:

“Several steps must behave as one indivisible unit.”

What volatile Is Bad For

Do not use `volatile` as if it solved:

- incrementing a shared counter safely
- protecting a check-then-act sequence
- safely updating multiple variables as one transaction
- replacing locks in all designs

Common student mistake

They see “shared variable” and immediately think:

```
make it volatile
```

That is often incomplete or wrong.

The Famous Trap: volatile Counter

```
1      class VolatileCounter {  
2          static volatile int counter = 0;  
3  
4          static void increment() {  
5              counter++;  
6          }  
7      }
```

Question

Why is this still unsafe?

Why counter++ Is Not Atomic

The statement

`counter++`

is not one indivisible action. Conceptually, it is:

```
1      int temp = counter;  
2      temp = temp + 1;  
3      counter = temp;
```

Two threads can interleave these steps and lose updates.

Key point

Even if each read and write is visible, the *whole update* is still a multi-step race.

Visibility Does Not Equal Atomicity

Visibility question

Did thread B see thread A's latest write?

Atomicity question

Did the update behave as one indivisible operation?

Critical distinction

`volatile` helps with the left question. It does not solve the right question.

This is why Part 3 exists

Next we will separate visibility problems from atomicity problems more sharply.

A Comparison Slide

Mechanism	Visibility	Ordering	Atomicity
Plain field	No guarantee	No guarantee	No guarantee for compound action
volatile	Yes	Yes	No Yes for protected critical section
synchronized / lock	Yes	Yes	Yes for protected critical section

Why this slide matters

Students must stop treating all concurrency tools as interchangeable.

Another Practical Example: Configuration Publication

```
1      class ConfigDemo {
2          static String config;
3          static volatile boolean loaded = false;
4
5          public static void main(String[] args) {
6              Thread loader = new Thread(() -> {
7                  config = "MODE=SAFE";
8                  loaded = true;
9              });
10
11              Thread user = new Thread(() -> {
12                  while (!loaded) {
13                      }
14                  System.out.println(config);
15              });
16
17              user.start();
18          }
19      }
```

When volatile Is Elegant

Elegant case

A thread prepares a result once, then tells other threads:

"It is ready now."

This is where `volatile` shines:

- simple
- efficient
- no lock-based waiting needed for the flag itself

But elegance has limits

As soon as your correctness depends on a multi-step shared update, `volatile` alone is usually not enough.

Common Misconceptions to Kill Early

Students should **not** leave this lecture thinking:

- “volatile means thread-safe”
- “volatile is a lighter lock”
- “volatile fixes every shared variable problem”
- “if a value is visible, then increments are safe”

Replace with

volatile is a tool for visibility and ordering of shared communication, not a general solution for compound shared-state correctness.

Part 2 Concept Summary

<code>volatile</code>	special shared field with visibility and ordering guarantees
Visibility	later reads can observe the written value correctly
Ordering	earlier writes become meaningful relative to the volatile signal
Happens-before	formal guarantee relation between memory actions
Good use	flags, readiness, publication
Limitation	no compound atomicity

Lab Preparation for Part 2

Lab 4.2 — Fix the Stop Flag

Students convert an unsafe stop flag into a volatile stop flag.

Lab 4.3 — Publication with a Volatile Flag

Students publish ordinary data using:

- data field
- volatile ready flag

Lab 4.4 — Volatile Counter Failure

Students test a volatile counter under repeated increments and explain why the result is still wrong.

Suggested Discussion Questions

- ① Why does `volatile` fix the stop flag?
- ② Why is `volatile` useful for publication patterns?
- ③ What is the difference between seeing a value and updating safely?
- ④ Why is clock time not enough to reason about memory visibility?
- ⑤ Why does `counter++` remain unsafe even when `counter` is `volatile`?

Takeaway

Final takeaway for Part 2

`volatile` allows threads to communicate through a variable with defined visibility and ordering semantics.

But

It does *not* make compound shared-state operations atomic.

Bridge to Part 3

Now that we know how visibility can be fixed, the next question is:

Why can a program still be wrong even when visibility is correct?

That leads directly to:

Questions?